

A Handwritten Music Recognition System Based on Visual Object Detection and semantic Assembly

Yuguang Yao Second Author

December 14, 2025

Abstract

Optical Music Recognition is a field of research that investigates how to computationally read music notation in documents [2]. This project proposes an end-to-end recognition pipeline, inspired by a precedent work [4]. We first reproduce the original system. The system leverages an advanced deep learning object detection model (YOLOv12) to extract basic musical symbols from score images, and combines a MLP module to identify the relationships between symbols with a constructed benchmark to evaluate the performance of the system. To realize a full OMR functionality, we expand this system with a rule-based assembly module to a ready-to-use musicXML file. Specifically, this assembly module resolves complex musical structure reconstruction issues, including multi-staff grouping, polyphonic alignment, and tuplet detection.

Keywords— Optical Music Recognition, YOLOv12, MLP, Scoretree, MusicXML

1 Introduction

Optical Music Recognition (OMR) is defined as a research field that investigates how to computationally read music notation in documents, aiming to invert the music encoding process to recover both musical semantics and structured encoding [2]. Unlike Optical Character Recognition (OCR), which typically deals with linear sequences of text, OMR must handle a complex visual language where the goal is often to enable replayability or detailed score editing. This requires the system not only to identify symbols but also to interpret their specific musical meaning, such as pitch and duration, which distinguishes OMR as a unique challenge separate from standard graphics recognition.

The primary difficulties in OMR stem from the two-dimensional and contextual nature of music notation, where the meaning of a symbol depends heavily on its spatial relationship with other elements, such as clefs and staff lines, rather than just its shape. Visual processing is further complicated by the extreme variability in symbol sizes, ranging from tiny dots to large braces, and the frequent superimposition or complex alignment of objects

like beams and slurs. Recent work has shown that while Convolutional Recurrent Neural Networks (CRNN) can be applied to OMR in an end-to-end fashion, their application is not straightforward due to the presence of different elements sharing the same horizontal position, particularly in homophonic scores where multiple voices occur simultaneously [1]. Additionally, the field faces significant hurdles due to the lack of standardized evaluation metrics and output formats, making it difficult to objectively assess the performance of OMR systems or compare their ability to fully reconstruct a score.

Recent advances in end-to-end OMR have focused on bridging the gap between recognition outputs and practical music notation formats. Mayer et al. [3] proposed a linearized MusicXML encoding that maintains compatibility with industry-standard formats while enabling end-to-end training for pianoform music. Building on this direction, Yang et al. [4] presented a more complete OMR solution that combines deep learning object detection with graph-based relationship modeling, demonstrating the effectiveness of separating visual recognition from semantic assembly. Their work serves as a key inspiration for our approach.

Our project aims to solve this challenge by combining deep learning visual recognition with symbolic logical reasoning. We propose a multi-stage processing framework: visual perception, structure construction, semantic reconstruction, and digital generation. The core contributions of this system are:

1. Proposed a high-precision handwritten music symbol detection scheme based on **YOLOv12**.
2. Designed a MLP-based score semantic assembly algorithm capable of handling complex symbol dependencies.
3. Implemented a complete conversion flow from image to MusicXML, supporting polyphony and complex rhythms.

2 System Pipeline

The core architecture of this project consists of a sequential processing pipeline: Dataset Preparation, Preprocessing, Visual Object Detection, Relation Prediction, Semantic Assembly, and MusicXML Generation. Each stage is

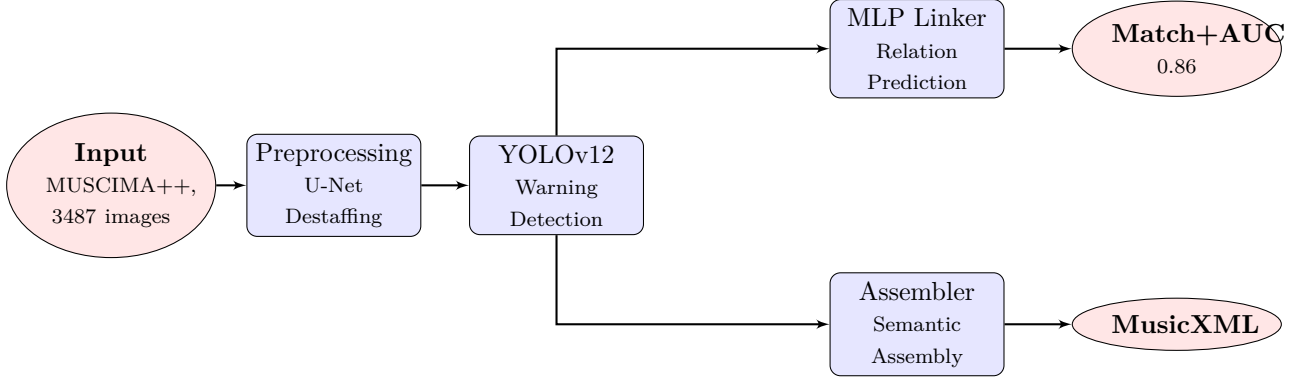


Figure 1: System Data Flow: The pipeline processes the input image through visual detection (YOLO), relation inference (MLP Linker), and rule-based assembly to generate the final MusicXML.

described below with implementation details and experimental results.

2.1 Dataset

We evaluate our system on the **MUSCIMA++** dataset, which contains 140 pages of complex handwritten music with 3487 cropped images. The dataset provides ground-truth annotations for both symbol bounding boxes and their relationships (Notation Graph). We follow the standard train/validation/test split logic to ensure rigorous evaluation. The dataset covers 117 symbol classes including noteheads, stems, beams, rests, accidentals, clefs, and time signatures.

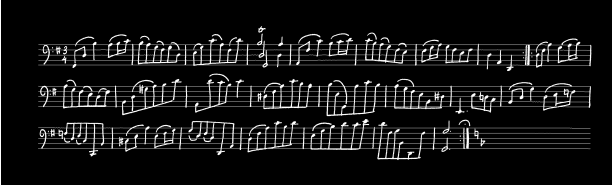


Figure 2: Example input image from MUSCIMA++ dataset showing handwritten musical notation.

2.2 Data Preprocessing

To optimize the input for object detection, we employ a **U-Net** architecture for image preprocessing. U-Net is a convolutional network with an encoder-decoder structure and skip connections, originally designed for biomedical image segmentation. In our pipeline, it is trained to perform pixel-wise classification, effectively separating musical symbols from staff lines and background noise.

We processed the raw data as follows:

1. **Staff Removal:** Leveraging the U-Net output, we remove the staff lines that often overlap with and obscure musical symbols. This "destaffing" significantly improves the signal-to-noise ratio for the subsequent YOLO detector.

2. **Image Cropping:** High-resolution full-page scores are sliced into local patches suitable for YOLO model input (typically 640×640). We employed an overlapping strategy during slicing to prevent key symbols from being cut off at the boundaries.

3. **Annotation Conversion:** MUSCIMA++ XML notation graph annotations were converted into YOLO format for training.



Figure 3: Result after U-Net preprocessing: staff lines removed, symbols preserved.

2.3 Visual Object Detection (YOLOv12)

We adopt the **YOLO (You Only Look Once)** series models as the visual perception engine. The project utilizes the latest **YOLOv12** architecture. Compared to YOLOv8, v12 significantly improves detection accuracy for small objects (such as augmentation dots and accidentals) while maintaining real-time performance.

Implementation Details: The YOLOv12 model is trained for 100 epochs with SGD optimizer. Input resolution is set to 640. The YOLO model is trained to minimize a composite loss function including bounding box regression and classification error. We utilize Mosaic data augmentation to enhance the model's robustness against scale variations and partial occlusions.

Results: The YOLOv12 model achieves high mAP scores on common symbols (Noteheads, Clefs), ensuring a solid foundation for subsequent assembly stages.



Figure 4: YOLOv12 detection results with bounding boxes around identified musical symbols.

2.4 Relation Prediction (MLP Linker)

After detecting discrete symbols, the system must reconstruct their relationships (e.g., which notehead is attached to which stem). We model this as a link prediction problem on a fully connected graph where each detected symbol is a node.

Feature Engineering: For a pair of symbols (u, v) , the model extracts a comprehensive feature vector combining geometric and semantic information:

- **Geometric Features:** Relative distance ($d_x = x_v - x_u$, $d_y = y_v - y_u$), normalized bounding box coordinates, and overlap ratios including IoU (Intersection over Union). These features capture the spatial relationships critical for determining valid connections.
- **Semantic Features:** Learned embedding vectors $\mathbf{e}_u, \mathbf{e}_v \in \mathbb{R}^{32}$ corresponding to the class IDs of symbols u and v . The embeddings are concatenated as $[\mathbf{e}_u; \mathbf{e}_v]$ to form a 64-dimensional semantic representation, allowing the model to learn class-specific connection patterns (e.g., noteheads typically connect to stems, but not to clefs).

The MLP outputs a probability score $P(u \rightarrow v) \in [0, 1]$ indicating the likelihood of a directed edge existing between them. This allows us to construct a **Notation Graph** where nodes are primitives and edges represent semantic connections.

Implementation Details: The MLP consists of 3 hidden layers with ReLU activation. It is trained to predict adjacency matrices from symbol pairs.

Evaluation Metrics: We adopt the **Match+AUC** metric proposed by Yang et al. [4] to evaluate the MLP Linker’s performance. This metric combines two complementary aspects:

- **Match Score:** Measures the precision of predicted edges against ground truth notation graphs. For each predicted edge (u, v) , we verify if it exists in the ground truth graph, computing the ratio of correct predictions.
- **AUC (Area Under Curve):** Evaluates the ranking quality of edge probabilities. A higher AUC indicates better discrimination between true and false edges across different threshold values.

Results: The MLP Linker achieves a **Match+AUC score of 0.86** on the test set, demonstrating that our

model effectively learns the geometric and semantic patterns of symbol relationships. It successfully filters out valid connections from the quadratic number of possible pairs. The conceptual ”overfitting” ensures that for the limited vocabulary of a single page, the connectivity is robust.

2.5 Semantic Assembly

This module transforms the Notation Graph into a hierarchical **Score Tree** suitable for MusicXML generation. The assembly pipeline consists of six sequential phases that progressively build musical structure from geometric primitives.

2.5.1 Phase 1: System & Staff Grouping

Goal: Partition staff lines $L = \{l_1, l_2, \dots\}$ into *Systems* (musical rows) and assign *Part* IDs (voices).

Staff Clustering: We apply a priority-based grouping strategy:

1. **Brace/Bracket Detection** [Highest Priority]: For each symbol g with class `multi-staff_brace` or `staff_grouping`, identify all staff lines l_i where the Y-axis overlap ratio $Overlap_Y(g, l_i) > 0.5$. Group these lines into a single System.
2. **Measure Separator Detection:** For `measure_separator` symbols spanning multiple staves vertically, group all touched staff lines.
3. **Fallback:** Ungrouped staff lines become independent single-staff Systems.

Part Indexing: Within each System, sort staff lines by Y-coordinate (top-to-bottom) and assign sequential Part IDs: $Part_1, Part_2, \dots$

2.5.2 Phase 2: Measure Slicing

Goal: Horizontally segment each System into Measures using barline positions.

Barline Validation: For each candidate barline symbol, we enforce geometric constraints:

- Aspect ratio: $width/height < 0.3$ (barlines are narrow)
- Minimum height: $height \geq 20$ pixels
- Staff overlap: Must overlap $\geq 50\%$ of staff system height

X-Axis Projection Fusion: Due to detection noise, barlines at the same temporal position may have slightly different X-coordinates. We merge barlines where $|x_i - x_j| < \max(\frac{w_{notehead}}{2}, 15)$ pixels, averaging their X-positions to define *Global Barlines*.

Bucket Sort: Based on Global Barline positions $(x_1, x_2, \dots, x_n)$, symbols are partitioned into measures: symbols with $x_i \leq x_{center} < x_{i+1}$ belong to Measure i .

2.5.3 Phase 3: Symbol Assembly (Note Construction)

Goal: Link noteheads, stems, beams, flags, dots, and accidentals into complete note objects.

Notehead-to-Stem Linking: For each notehead:

- **Filled Noteheads** (notehead-full):
 1. If overlapping stem found, link to it.
 2. Otherwise, create a *virtual stem*: Search upward/downward for beams/flags within half the inter-system distance. If found, extend virtual stem to reach them. If not, create default-length stem ($3.5 \times \text{line_spacing}$) with direction determined by notehead position relative to middle staff line.
- **Other Noteheads:** Link to overlapping stem or nearest stem.

Stem-to-Beam/Flag Linking: For each stem, find all beams/flags within stem.width distance. Return ALL qualifying beams/flags (not just the first).

Dot Detection: For each notehead, search rightward for duration-dot within distance thresholds ($\Delta x < 30px$, $\Delta y < 30px$). If found, multiply duration by 1.5.

2.5.4 Phase 4: Attribute Recognition

State Persistence Principle: Musical attributes (clef, key, time signature) are inherited from previous measures unless explicitly changed.

Clef Detection: Search for g-clef, f-clef, c-clef. If found, update; if absent, inherit. First measure defaults: Part 1 = Treble, Part 2 = Bass.

Key Signature: Count sharp/flat symbols in the region right of clef, left of time signature. Cluster symbols with $\Delta x < 1.5 \times \text{note_width}$ as a single key signature group.

Time Signature: Detect time_signature label or vertical digit stacks (e.g., numeral_3 above numeral_4 indicates 3/4).

2.5.5 Phase 5: Triplet Detection

We employ a *cascaded rule set* with decreasing constraint strength:

Rule 1 - Bracket-Based [Highest Priority]: For each `tuplet_bracket`, find notes whose notehead centers fall within bracket's X-range $[x_{start}, x_{end}]$. Filter out notes with vertical distance $> 1.5 \times \text{staff_height}$. If exactly 3 notes remain, mark as triplet (even without `numeral_3`).

Rule 2 - Beam-Based: For beams linking $3k$ stems (where $k \in \mathbb{N}$), search for `numeral_3` within radius $3 \times \text{line_spacing}$ of beam center. *Finger Number Exclusion:* Reject if `numeral_3` is vertically aligned above a single notehead (likely a finger number). Accept if centered over the note group.

Rule 3 - Numeral-3-Centered: For each unused `numeral_3` (excluding time signature digits), compute Euclidean distances to all notes within radius $6 \times \text{line_spacing}$. Select the 3 closest notes. Apply finger number exclusion logic.

Rule 4 - Triplet Modification: For confirmed triplet group $G = \{n_1, n_2, n_3\}$:

- Set $\text{duration}_{xml} = \text{base_duration} \times \frac{2}{3}$
- MusicXML tags: n_1 gets `<triplet type="start"/>`, n_3 gets `<triplet type="stop"/>`

Rule 5 - Sanity Check [Fallback]: If $\sum \text{note.duration} > \text{time_signature_duration} + \text{tolerance}$, search for consecutive 3-note groups with identical `base_duration`. If converting to triplet fixes the overflow and no `numeral_3` exists in that region, force conversion with low confidence.

2.5.6 Phase 6: Pitch Determination

Algorithm: Given notehead at Y-position y_{note} and staff lines $\{y_0, y_1, \dots, y_4\}$ (top to bottom):

1. Find closest staff line $y_{closest}$ and its index $i_{closest}$.
2. Calculate half-spacing: $h = \text{median}(\{y_{i+1} - y_i\})/2$
3. Compute steps from top line: $\text{steps} = i_{closest} \times 2 + \frac{y_{note} - y_{closest}}{h}$
4. Round to nearest 0.5 (half-step precision).
5. Map to diatonic scale using clef-specific reference:
 - Treble Clef: Top line = F5
 - Bass Clef: Top line = A3
 - Alto Clef: Top line = G4
6. Apply local accidentals, then key signature accidentals.

Example: In Treble Clef, if $\text{steps} = 2.0$, starting from F5, move down 2 diatonic steps: $F5 \rightarrow E5 \rightarrow D5$.



Figure 5: Semantic assembly result showing reconstructed relationships between musical symbols.

2.6 MusicXML Generation

The final Assembler successfully generates MusicXML files that can be opened in MuseScore and other music notation software. The system correctly reconstructs polyphonic voices and complex triplets using the defined rules. Figure 1 shows the complete data flow from input image to final MusicXML output.



Figure 6: Final MusicXML output rendered as musical notation, demonstrating successful end-to-end reconstruction.

3 Conclusion and Future Work

Our system demonstrates the powerful potential of combining computer vision with symbolic logic. By treating sheet music as a graph structure, we effectively captured the complex two-dimensional dependencies between musical symbols, successfully achieving the digital reconstruction of handwritten music.

An improvement direction can be following areas:

- **improve linker model:** The current linker model shows a high benchmark performance, but less useful in data due to illogical mismatches. And current rule-based reasoning is not enough to handle complex cases. We can try to use small models to help some subprocesses of rule-based reasoning.
- **More Symbol Types:** So far we only consider the essential symbols, we will try to include more symbols to improve the performance.
- **probabilistic and interactive processing:** Here we use yolo to detect the symbols we a threshold, but we can implement an interface that allows users to modify the recognition results with a probability output like Audervis.

References

- [1] María Alfaro-Contreras, José M. Iñesta, and Jorge Calvo-Zaragoza. Optical music recognition for homophonic scores with neural networks and synthetic music generation. *International Journal of Multimedia Information Retrieval*, 12(1):12, May 2023.
- [2] Jorge Calvo-Zaragoza, Jan Hajič Jr., and Alexander Pacha. Understanding optical music recognition. *ACM Computing Surveys*, 53(4):1–35, July 2020.
- [3] Jiří Mayer, Milan Straka, Jan Hajič, and Pavel Pecina. *Practical End-to-End Optical Music Recognition for Pianoform Music*, page 55–73. Springer Nature Switzerland, 2024.

- [4] Guang Yang, Muru Zhang, Lin Qiu, Yanming Wan, and Noah A. Smith. Toward a more complete omr solution, 2024.